

SMC Reset Unit — Low-Level Design

SMC Reset Unit — Low-Level Design (Internal)

1. Source structure
2. Process structure
 - Single-driver discipline
3. Reset derivation (`derive()`)
4. Register state & reset domains
5. FLR cool-reset sequencing
6. CCI parameter catalogue
7. TLM-2.0 callbacks
8. Future work

SMC Reset Unit — Low-Level Design (Internal)

This document describes the **internal SystemC implementation** of the SMC Reset Unit LT model: process structure, single-driver discipline, reset-domain handling, FLR sequencing, and the CCI parameter catalogue. The external contract is in `01_RESET_UNIT_Specification.md`.

1. Source structure

File	Contents
<code>include/reset_unit.h</code>	<code>reset_ctrl_t</code> , <code>jtag_reset_ctrl</code> , <code>reset_unit_cfg</code> , <code>SC_MODULE(reset_unit)</code> + <code>cci_param</code> declarations.
<code>src/reset_unit.cpp</code>	Constructor, processes, register decode, reset derivation, FLR sequencing, debug API.
<code>include/smc_tlm_extensions.h</code>	Shared <code>smc_axi_extension</code> GP extension (standalone build).

The RTL's structural split (`smc_reset_ctrl` / `smc_subsystem_resets` / `smc_cool_reset_wrap` / `smc_reset_sync`) is collapsed into a single `SC_MODULE`, since an LT model has no need for the four separate clock domains.

2. Process structure

Process	Kind	Sensitivity	Role
<code>input_method</code>	SC_METHOD	all input ports	Track <code>ss_reset_complete_i</code> ; detect FLR rising edge; call <code>process_state_change()</code> .
<code>flr_assert_method</code>	SC_METHOD	<code>flr_assert_event_</code>	Drive <code>flr_cool_n_low</code> (cool reset asserted).
<code>flr_deassert_method</code>	SC_METHOD	<code>flr_deassert_event_</code>	Drive <code>flr_cool_n_high</code> (cool reset released).
<code>output_method</code>	SC_METHOD	<code>recompute_event_</code>	Sole driver of every output port.

All input/output processes are `dont_initialize()`d; `start_of_simulation()` seeds the reset-edge trackers from the initial pin levels and posts the first `recompute_event_` so the outputs reach their reset-correct values at $t=0$.

Single-driver discipline

`output_method` is the only process that writes any `sc_out`. Every other code path (register write, FLR edge, JTAG override, input change) mutates internal state and calls `schedule_recompute()`, which posts `recompute_event_` at `SC_ZERO_TIME`. This satisfies SystemC 3.0's strict single-driver rule and mirrors the PLIC / CLINT models. Output writes are **idempotent** for the per-subsystem bundles (cached in `ss_ctrl_cache_`) to avoid spurious value-changed events on the 32 struct signals.

3. Reset derivation (`derive()`)

`derive()` is a pure function of the current input ports, the JTAG override bundle, and `flr_cool_n`. It reproduces the `smc_reset_ctrl.sv` combinational equations exactly (see spec §5) but **abstracts the timing elements**:

RTL element	LT abstraction
32-cycle cold-reset de-glitch shift register	zero latency
255-cycle cold-reset extender counter	zero latency
32-cycle powergood stretch	<code>powergood_stable = powergood_i</code>
32-cycle cool-from-pin de-glitch	zero latency
<code>prim_sync_reset</code> (4-deep) synchronisers	zero-delay pass-through

These are CDC / glitch-filtering details that do not change the functional (firmware-observable) reset relationships.

The `derived_t` struct returned by `derive()` carries the four levels the rest of the model needs: `powergood_stable`, `stable_cold_rst_n` (= cold domain), `rst_primary_n` (= primary domain), `rst_core_int_n`, `rst_wdt_n`.

4. Register state & reset domains

Registers are plain `uint32_t` members grouped by reset domain (`clear_primary_regs()` / `clear_cold_regs()`). `process_state_change()`:

1. Calls `derive()`.
2. On a **falling edge** of `rst_primary_n` (tracked via `prev_primary_n_`), clears the primary-domain group (`SS_WARM_RESET_N` → `0xFFFFFFFF`, rest 0).
3. On a **falling edge** of `stable_cold_rst_n` (tracked via `prev_cold_n_`), clears the cold-domain group. `ISOLATE_REQ_PINEN_REG` is cleared only when `isolate_req_pin_i` is low (the RTL gates its reset with the pin).
4. Updates the trackers and calls `schedule_recompute()`.

Because `rst_primary_n` depends on `stable_cold_rst_n`, a cold reset asserts both domains; a cool/FLR reset (driven through `stable_cool_rst_n` / `flr_cool_n`) asserts only the primary domain, so the FLR / isolate registers survive — exactly as the RTL retains the FLR counters across a cool reset.

`process_state_change()` is invoked from `input_method`, the two FLR methods, and `set_jtag_ctrl()`, so a reset assertion arising from *any* of those sources clears the right registers.

Known simplification. Register writes are not blocked while a reset is asserted (the RTL's register block ignores writes during `arst_n` low). Firmware does not write the block during reset, so the LT model permits it; documented here as a future-work item if strict write-gating is required.

5. FLR cool-reset sequencing

`input_method` detects the `cfg_flr_pf_active_i` rising edge (`prev_flr_active_`), sets `ISOLATE_REQ_SMC_REG.bit0`, and calls `flr_kick()`:

```
if (FLR_RESET_COUNTER_VALUE == 0) return;           // RTL: 0 ⇒ no flow
flr_assert_event_.notify(ref_clk_period × FLR_COUNTER_VALUE);
flr_deassert_event_.notify(ref_clk_period × (FLR_COUNTER_VALUE + FLR_RESET_COUNTER_VALUE));
```

`flr_assert_method` / `flr_deassert_method` then drive `flr_cool_n` low/high and re-run `process_state_change()` (so the cool reset both pulses `rst_cool_no` and asserts the primary domain). This collapses the RTL's two `prim_updown_counter` + FSM pair into two timed events while preserving the observable pulse shape and timing (in `ref_clk_period_ns` units).

6. CCI parameter catalogue

Name	Type	Default	Mutability	Notes
<code>num_subsystems</code>	unsigned	32	CCI_IMMUTABLE	Sizes <code>ss_reset_ctrl_o[]</code> ; validated 1..32.
<code>ref_clk_period_ns</code>	double	10.0	CCI_IMMUTABLE	FLR pulse time base; validated ≥ 0 .
<code>access_delay_ns</code>	double	2.0	mutable	Re-read every <code>b_transport</code> ; validated ≥ 0 .

CCI params are declared **before** the public ports in the header so they are constructed first and `ss_reset_ctrl_o` can be sized from the (possibly preset-overridden) `num_subsystems`. Provenance metadata (`rdl_dimension`, `valid_range`, `unit`, `rtl_signal`, `tlm_phase`) is attached for inspector tooling. Misconfiguration (`num_subsystems` $\notin$ 1..32, negative periods) raises `SC_REPORT_FATAL` at construction.

7. TLM-2.0 callbacks

`b_transport` validates width (4 B), alignment, streaming width, and byte-enable absence, then window membership, before dispatching to `reg_read` / `reg_write` and annotating the delay with `access_delay_ns`. `transport_dbg` shares the decode but skips the delay and applies the same register semantics without

simulation side effects (it does not run the FLR edge detector). DMI is never granted.

8. Future work

- Strict register write-gating during reset assertion.
- Optional cycle-accurate de-glitch / extender / stretch counters behind a CCI flag, for users who need the exact reset-stretch timing.
- VCD tracing helper for the `reset_ctrl_t` bundle signals.